

A project report on

# **BANTR: AI-POWERED VOICE DEBATE PLATFORM**

Submitted in partial fulfillment for the award of the degree of

**Bachelor of Technology**

by

**NAME OF THE CANDIDATE (REG. NO.)**

**School of Computer Science and Engineering**

**VIT-AP University**

April 2026

# **BANTR: AI-POWERED VOICE DEBATE PLATFORM**

Submitted in partial fulfillment for the award of the degree of

**Bachelor of Technology**

by

**NAME OF THE CANDIDATE (REG. NO.)**

**School of Computer Science and Engineering**

**VIT-AP University**

April 2026

## DECLARATION

I hereby declare that the project report entitled "Bantr: AI-Powered Voice Debate Platform" is a record of work carried out for the Senior Design Project. The work presented in this report describes the design and implementation of a voice-based debate application with authenticated users, LiveKit rooms, an AI debate worker, transcript analysis, embeddings, and chat memory.

I further declare that this work has not been submitted, either in part or in full, for the award of any other degree or diploma in this institute or any other institute or university.

Place: Amaravati

Date: April 2026

Signature of the Candidate

## CERTIFICATE

This is to certify that the Senior Design Project titled "Bantr: AI-Powered Voice Debate Platform" submitted by NAME OF THE CANDIDATE (REG. NO.) is in partial fulfillment of the requirements for the award of the degree of Bachelor of Technology at VIT-AP University.

The report presents a working proof of concept for authenticated voice debates against an AI opponent, supported by transcript persistence, structured performance analysis, vector-searchable memory, and contextual coaching chat.

Internal Guide

The thesis is satisfactory / unsatisfactory

Internal Examiner

External Examiner

Approved by

Dean

School of Computer Science and Engineering

## ABSTRACT

Bantr is a voice-based debate platform designed to help users practice argumentation with an AI opponent and receive actionable feedback after each session. The system allows an authenticated user to create a debate topic, configure an AI opponent prompt and voice, join a real-time LiveKit room, end the debate, review the transcript, generate structured analysis, and continue learning through a chatbot that remembers relevant moments from previous debates.

The project is implemented as a FastAPI backend, a React TypeScript frontend, a standalone LiveKit agent worker, and a PostgreSQL database with pgvector. LiveKit handles WebRTC transport and AI voice room orchestration. OpenAI is used for structured debate analysis, embeddings, and retrieval-augmented coaching responses.

The final product delivers the core proof-of-concept workflow: debate creation, room start and end, transcript storage, analysis generation, embedding persistence, contextual chat, health checks, demo runbooks, and automated smoke-test assets. The design focuses on correctness, security boundaries, clear lifecycle states, and reliable integration points.

## ACKNOWLEDGEMENT

I express my sincere gratitude to the project guide, faculty members, and the School of Computer Science and Engineering, VIT-AP University, for their guidance and support during the design and development of this Senior Design Project.

I also thank the maintainers of the open-source technologies used in this project, including FastAPI, React, SQLAlchemy, Alembic, LiveKit, OpenAI SDKs, pgvector, pytest, Ruff, Pyright, Vite, and Playwright.

Finally, I thank my peers and reviewers for feedback on the application workflow, clarity of the user experience, and practical readiness of the final demo.

## CONTENTS

| Section                                    | Page |
|--------------------------------------------|------|
| Abstract                                   | 5    |
| Acknowledgement                            | 6    |
| List of Tables                             | 8    |
| List of Figures                            | 9    |
| List of Acronyms                           | 10   |
| Chapter 1 Introduction                     | 11   |
| Chapter 2 Literature and Technology Review | 12   |
| Chapter 3 Requirements and Analysis        | 13   |
| Chapter 4 System Design                    | 15   |
| Chapter 5 Implementation                   | 17   |
| Chapter 6 Testing and Results              | 19   |
| Chapter 7 Conclusion and Future Work       | 21   |
| References                                 | 22   |
| Appendices                                 | 23   |

## LIST OF TABLES

| Table No. | Title                       |
|-----------|-----------------------------|
| 1.1       | Project scope summary       |
| 2.1       | Technology stack            |
| 3.1       | Functional requirements     |
| 3.2       | Non-functional requirements |
| 4.1       | Database entities           |
| 5.1       | API endpoint summary        |
| 6.1       | Quality gates               |

## LIST OF FIGURES

| Figure No. | Title                       |
|------------|-----------------------------|
| 1.1        | Bantr end-to-end workflow   |
| 4.1        | System architecture         |
| 4.2        | Debate lifecycle state flow |
| 5.1        | Frontend route map          |

## LIST OF ACRONYMS

| Acronym | Expansion                         |
|---------|-----------------------------------|
| AI      | Artificial Intelligence           |
| API     | Application Programming Interface |
| CSRF    | Cross-Site Request Forgery        |
| CRUD    | Create, Read, Update, Delete      |
| JWT     | JSON Web Token                    |
| LLM     | Large Language Model              |
| PoC     | Proof of Concept                  |
| RAG     | Retrieval-Augmented Generation    |
| STT     | Speech to Text                    |
| TTS     | Text to Speech                    |
| VAD     | Voice Activity Detection          |
| WebRTC  | Web Real-Time Communication       |

# CHAPTER 1 INTRODUCTION

## 1.1 Project Overview

Bantr is a learning-focused debate application that lets a user debate a configurable AI opponent in a live voice room. The product combines real-time conversation with post-session learning artifacts: transcript, structured analysis, vector memory, and a coaching chatbot.

## 1.2 Problem Statement

Most debate practice tools either provide static prompts or text-only feedback. They do not reproduce the pressure of speaking in real time, and they rarely preserve previous debates as searchable learning context. Bantr addresses this gap by combining real-time voice interaction with persistent analysis and memory-backed coaching.

## 1.3 Objectives

- Provide secure user authentication and user-owned debate data.
- Allow users to create debate topics and configure an AI opponent with a prompt and voice.
- Run a real-time voice debate using LiveKit WebRTC rooms and an AI agent worker.
- Persist debate transcripts and analyze them with structured scoring and feedback.
- Store transcript embeddings in pgvector so a chatbot can answer questions using past debate context.
- Deliver a usable React frontend and documented backend workflow for local demo validation.

## 1.4 Scope

| In Scope                                                 | Out of Scope                                        |
|----------------------------------------------------------|-----------------------------------------------------|
| Single-user proof of concept with authenticated accounts | Multi-user debates and tournaments                  |
| AI voice debate room through LiveKit                     | Audio file recording and playback archive           |
| Transcript, analysis, embeddings, and coaching chat      | Real-time streaming analysis during the debate      |
| Local/demo readiness with smoke tests and runbooks       | Horizontal scaling and production rate-limit design |

## 1.5 Expected Users

The primary user is a student or debate learner who wants repeated practice with an opponent that can be customized for different topics, viewpoints, and difficulty levels.

## CHAPTER 2 LITERATURE AND TECHNOLOGY REVIEW

### 2.1 Real-Time Voice Interaction

Voice debate requires low-latency transport, speech recognition, turn handling, response generation, and speech synthesis. LiveKit was selected because it provides WebRTC room infrastructure, participant tokens, and an agent framework that can connect AI services to real-time rooms.

### 2.2 Large Language Models for Feedback

LLMs are suitable for structured qualitative review when outputs are constrained by schemas and prompts. Bantr uses direct OpenAI calls for post-debate analysis so the backend can validate and persist fields such as argument strength, fallacies, persuasiveness, key moments, improvement areas, summary, and winner.

### 2.3 Vector Search and Memory

A debate assistant becomes more useful when it can refer to previous debates. Bantr chunks transcripts, embeds the chunks, stores them in PostgreSQL with pgvector, and retrieves similar chunks for a retrieval-augmented chat response.

### 2.4 Technology Stack

| Layer       | Technology                                        | Purpose                                                                          |
|-------------|---------------------------------------------------|----------------------------------------------------------------------------------|
| Frontend    | React 18, TypeScript, Vite, Tailwind              | Authenticated app pages, debate dashboard, live room, analysis, transcript, chat |
| Backend API | Python 3.12, FastAPI, Pydantic v2                 | HTTP API, validation, auth, CSRF, service orchestration                          |
| Persistence | PostgreSQL, SQLAlchemy async, Alembic, pgvector   | Relational data, migrations, transcript embeddings                               |
| Voice Rooms | LiveKit Cloud and LiveKit React components        | WebRTC rooms and frontend room connection                                        |
| AI Worker   | LiveKit Agents SDK                                | Standalone AI debate participant and transcript capture                          |
| AI Services | OpenAI, LiveKit inference plugins                 | Analysis, embeddings, debate LLM, STT, TTS                                       |
| Quality     | pytest, Ruff, Pyright, ESLint, Vitest, Playwright | Backend and frontend verification                                                |

## CHAPTER 3 REQUIREMENTS AND ANALYSIS

### 3.1 Functional Requirements

| ID  | Requirement                                                 | Implemented Through                               |
|-----|-------------------------------------------------------------|---------------------------------------------------|
| FR1 | Register, login, refresh, logout, and identify current user | Auth router, cookies, refresh tokens, CSRF checks |
| FR2 | Create, list, view, start, end, and delete debates          | Debates router, debate service, CRUD layer        |
| FR3 | Join an active voice debate room                            | LiveKit token generation and React LiveKit room   |
| FR4 | Persist debate transcript after room end                    | Standalone agent worker and transcript CRUD       |
| FR5 | Generate structured debate analysis                         | OpenAI analysis service and DebateAnalysis model  |
| FR6 | Persist vector embeddings for transcript chunks             | Embedding service and pgvector table              |
| FR7 | Chat with contextual memory from previous debates           | Chat service with vector retrieval and history    |
| FR8 | Expose integration readiness checks and demo smoke test     | Health endpoint and backend demo script           |

### 3.2 Non-Functional Requirements

| Category        | Requirement                                         | Design Choice                                                                       |
|-----------------|-----------------------------------------------------|-------------------------------------------------------------------------------------|
| Security        | Protect user-owned data and state-changing requests | Auth dependencies, ownership filtering, CSRF validation, 404 for wrong user objects |
| Reliability     | Avoid corrupt lifecycle states                      | Explicit debate status transitions and agent-owned completion                       |
| Maintainability | Keep backend responsibilities separated             | Routers, schemas, services, CRUD, models, and core modules                          |
| Validation      | Reject invalid request data at the backend          | Pydantic schemas and typed service boundaries                                       |
| Demo readiness  | Allow repeatable setup and validation               | SETUP guide, demo checklist, smoke-test script                                      |

### 3.3 Key Edge Cases

- Invalid or blank debate fields are rejected by backend schemas.
- Access to another user's debate returns not found instead of exposing object existence.
- Start is allowed only from pending status; end is allowed only from active status.
- Analysis requires a completed debate and an existing transcript; duplicate analysis returns conflict.

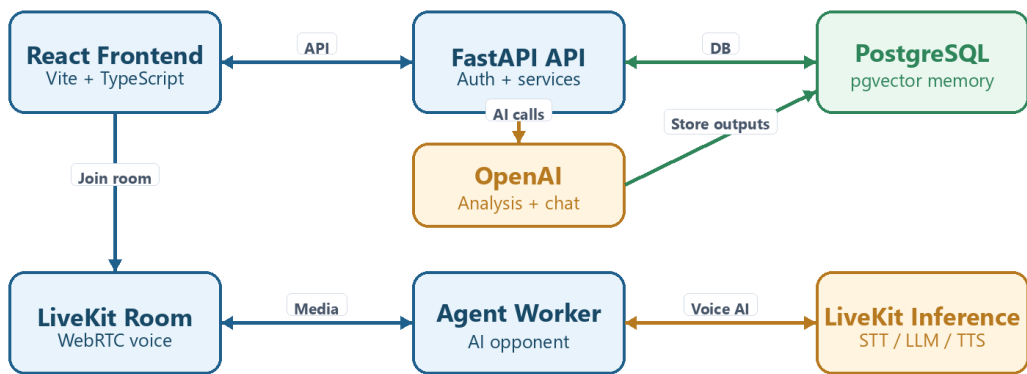
- OpenAI failures during analysis roll back the transaction to avoid partial analysis and embeddings.
- Deleted or missing debates referenced by old chat context are filtered from responses.
- LiveKit room creation failure marks the debate as failed and reports an upstream failure.
- Agent worker database sessions are manually committed or rolled back because the worker runs outside FastAPI dependency injection.

## CHAPTER 4 SYSTEM DESIGN

### 4.1 Architecture

Bantr follows a monolith plus sidecar worker architecture. The FastAPI process handles HTTP requests, authentication, database workflows, and external analysis calls. The LiveKit agent worker is a separate process that joins LiveKit rooms, operates the AI debate participant, and persists transcripts after sessions end.

#### Bantr System Architecture



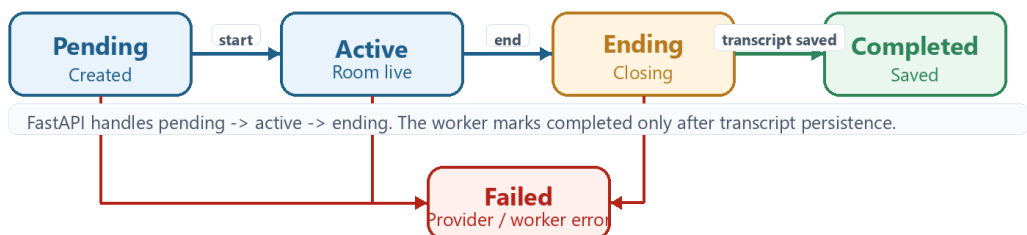
FastAPI owns product workflow; LiveKit carries voice; OpenAI and pgvector power feedback and memory.

### 4.2 Backend Layering

- API routers handle HTTP concerns, dependency injection, request schemas, response models, and status codes.
- Schemas define trusted input and output contracts using Pydantic v2.
- Services own business workflows such as debate lifecycle, analysis, embeddings, and chat.
- CRUD modules contain focused database reads and writes.
- Models define SQLAlchemy tables and relationships.
- Core modules centralize configuration, security, errors, logging, and auth utilities.

### 4.3 Debate Lifecycle

#### Debate Lifecycle State Flow



FastAPI handles pending -> active -> ending. The worker marks completed only after transcript persistence.

The API server owns pending to active and active to ending transitions. The agent worker owns ending to completed after transcript persistence. This avoids marking a debate complete before the transcript exists.

#### 4.4 Database Design

| Entity          | Purpose                                 | Important Fields                                                     |
|-----------------|-----------------------------------------|----------------------------------------------------------------------|
| User            | Authenticated account owner             | email, username, roles, refresh tokens                               |
| Debate          | Main debate configuration and lifecycle | user_id, title, topic, prompt, voice, status, room, timestamps       |
| Transcript      | One transcript per debate               | full_text, speaker_segments                                          |
| DebateAnalysis  | Structured performance feedback         | scores, fallacies, persuasiveness, key moments, improvements, winner |
| DebateEmbedding | Vector-searchable transcript chunks     | debate_id, user_id, chunk_index, chunk_text, speaker, vector         |
| ChatMessage     | Memory chat history                     | user_id, role, content, context_debate_ids                           |

#### 4.5 Frontend Design

The frontend is a React TypeScript application with pages for landing, authentication, dashboard, debate creation, debate details, live debate, transcript, analysis, chat, resources, settings, notifications, and admin users.

## CHAPTER 5 IMPLEMENTATION

### 5.1 Authentication and Security

Bantr uses cookie-based authentication with access and refresh tokens, CSRF protection for state-changing routes, Google OAuth support, role and permission models, and user ownership checks on debate and chat data.

### 5.2 Debate API

| Method | Endpoint                        | Purpose                                                         |
|--------|---------------------------------|-----------------------------------------------------------------|
| POST   | /api/v1/debates                 | Create a pending debate with server-generated LiveKit room name |
| GET    | /api/v1/debates                 | List debates owned by the current user                          |
| GET    | /api/v1/debates/{id}            | Read one owned debate                                           |
| POST   | /api/v1/debates/{id}/start      | Create LiveKit room, dispatch agent, return join token          |
| POST   | /api/v1/debates/{id}/end        | Delete LiveKit room and move debate to ending                   |
| DELETE | /api/v1/debates/{id}            | Delete debate and cascaded child data                           |
| GET    | /api/v1/debates/{id}/transcript | Return persisted transcript                                     |
| POST   | /api/v1/debates/{id}/analyze    | Generate analysis and embeddings                                |
| GET    | /api/v1/debates/{id}/analysis   | Return stored analysis                                          |

### 5.3 Agent Worker

The worker runs from backend/agent\_worker.py and is started independently from the API server. It reads room metadata, configures the LiveKit agent session, joins the debate room, generates the opening response, collects the conversation transcript, opens its own async database session, writes the transcript, and updates the debate state.

### 5.4 Analysis and Embeddings

After a debate is completed, the analysis service loads the transcript and calls OpenAI with a structured prompt. The resulting fields are validated and stored as a DebateAnalysis record. The embedding service chunks transcript content, creates OpenAI embeddings, and stores them in pgvector with debate and user identifiers for scoped retrieval.

## 5.5 Contextual Chat

The chat service embeds the user question, searches the user's transcript embeddings for the most relevant chunks, loads recent chat history, builds a concise debate-coach prompt, calls OpenAI, stores both user and assistant messages, and returns the assistant response with referenced debate metadata.

## 5.6 Frontend Workflow

1. The user signs in through email/password or Google OAuth.
2. The dashboard and My Debates pages show existing debates and status.
3. Create Debate collects title, topic, AI prompt, and voice settings.
4. Live Debate connects to the LiveKit room using the backend token.
5. Debate Detail, Transcript, and Analysis pages expose post-session artifacts.
6. Chat provides follow-up coaching using stored debate memory.

## CHAPTER 6 TESTING AND RESULTS

### 6.1 Verification Strategy

The project includes backend tests, frontend build and lint checks, frontend unit and e2e scripts, documentation linting, database migration checks, health checks, and a demo smoke-test script. External services such as LiveKit and OpenAI require environment credentials for full manual or integration validation.

| Gate              | Command                                                                           | Purpose                                     |
|-------------------|-----------------------------------------------------------------------------------|---------------------------------------------|
| Backend tests     | <code>cd bantr &amp;&amp; uv run pytest</code>                                    | Validate backend behavior and async paths   |
| Backend lint      | <code>cd bantr &amp;&amp; uv run ruff check backend</code>                        | Find Python style and correctness issues    |
| Backend format    | <code>cd bantr &amp;&amp; uv run ruff format --check backend</code>               | Ensure formatter compliance                 |
| Backend typecheck | <code>cd bantr &amp;&amp; uv run pyright</code>                                   | Check production backend typing             |
| Frontend build    | <code>cd bantr/frontend &amp;&amp; npm run build</code>                           | Typecheck and build React app               |
| Frontend lint     | <code>cd bantr/frontend &amp;&amp; npm run lint</code>                            | Find TypeScript/React lint issues           |
| Frontend unit     | <code>cd bantr/frontend &amp;&amp; npm run test:unit</code>                       | Run component and hook tests                |
| Frontend e2e      | <code>cd bantr/frontend &amp;&amp; npm run test:e2e</code>                        | Run browser smoke coverage                  |
| Docs lint         | <code>cd bantr/frontend &amp;&amp; npm run docs:lint</code>                       | Validate markdown docs                      |
| Demo smoke        | <code>cd bantr/backend &amp;&amp; uv run python scripts/demo_smoke_test.py</code> | Exercise auth and debate lifecycle API flow |

### 6.2 Demo Validation Flow

1. Start the FastAPI server with uvicorn.
2. Start the LiveKit agent worker in a second terminal.
3. Check `/health/live`, `/health/ready`, and `/health/integrations`.
4. Login and verify auth cookie and CSRF token.
5. Create a debate and start the LiveKit room.
6. End the debate and wait for transcript completion.
7. Run analysis and verify structured results.
8. Ask the chat assistant a question about the completed debate.
9. Confirm chat history persists and context debates are returned.

### 6.3 Final Result

The implemented product satisfies the defined proof-of-concept scope. It provides an authenticated user flow, a real-time voice debate path, backend-controlled lifecycle transitions, transcript persistence, structured analysis, pgvector-backed memory, contextual chat, frontend pages for the workflow, and documentation for setup and demo validation.

### 6.4 Known Constraints

- LiveKit and OpenAI flows require valid external credentials and cannot be fully verified offline.
- Analysis is synchronous and may take several seconds for long transcripts, which is acceptable for the proof of concept.
- The current scope is single-user practice; multi-participant debate rooms are not implemented.
- Audio recordings are not stored; the durable learning artifact is the transcript and derived analysis.

## CHAPTER 7 CONCLUSION AND FUTURE WORK

### 7.1 Conclusion

Bantr demonstrates a complete AI-assisted voice debate learning workflow. The product goes beyond a simple chatbot by placing the user in a live speaking environment, persisting the resulting transcript, generating structured feedback, and making past debates searchable through vector memory.

The project successfully integrates FastAPI, React, LiveKit, OpenAI, SQLAlchemy async, Alembic, and pgvector into one coherent application. The result is suitable for a final review demonstration and provides a clear base for future product development.

### 7.2 Future Work

- Add multi-user debate rooms with moderation and scoring.
- Move analysis and embeddings to a background queue for longer transcripts.
- Add richer analytics dashboards that track improvement over time.
- Support debate templates, difficulty presets, and rubric customization.
- Store optional audio recordings with explicit consent and retention controls.
- Add rate limiting and abuse detection around costly AI endpoints.
- Improve transcript timestamps and speaker diarization as LiveKit SDK support evolves.
- Add deployment runbooks, production monitoring, and backup procedures.

## REFERENCES

1. FastAPI Documentation. (2026). FastAPI framework documentation.  
<https://fastapi.tiangolo.com/>
2. LiveKit Documentation. (2026). LiveKit WebRTC and Agents documentation.  
<https://docs.livekit.io/>
3. OpenAI Documentation. (2026). OpenAI API platform documentation.  
<https://platform.openai.com/docs/>
4. SQLAlchemy Documentation. (2026). SQLAlchemy 2.0 async ORM documentation. <https://docs.sqlalchemy.org/>
5. pgvector Documentation. (2026). Open-source vector similarity search for PostgreSQL. <https://github.com/pgvector/pgvector>
6. React Documentation. (2026). React library documentation. <https://react.dev/>
7. Vite Documentation. (2026). Frontend build tooling documentation.  
<https://vite.dev/>
8. Alembic Documentation. (2026). Database migration tool for SQLAlchemy.  
<https://alembic.sqlalchemy.org/>
9. Bantr repository documentation. (2026). Setup guide, demo checklist, engineering standards, and Bantr debate platform design specification.

## APPENDICES

### Appendix 1: Environment Variables

- DATABASE\_USER, DATABASE\_PASSWORD, DATABASE\_HOST, DATABASE\_PORT, DATABASE\_NAME
- GOOGLE\_CLIENT\_ID, GOOGLE\_CLIENT\_SECRET, GOOGLE\_REDIRECT\_URI
- LIVEKIT\_URL, LIVEKIT\_API\_KEY, LIVEKIT\_API\_SECRET
- OPENAI\_API\_KEY, OPENAI\_COMPLEX\_MODEL, OPENAI\_SIMPLE\_MODEL, LIVEKIT\_LLM\_MODEL, OPENAI\_EMBED\_MODEL
- FRONTEND\_URL, ALLOWED\_ORIGINS, SECRET\_KEY, MODE

### Appendix 2: Repository Files Used for Review

- docs/superpowers/specs/2026-04-15-bantr-debate-platform-design.md
- docs/superpowers/plans/2026-04-15-bantr-debate-platform.md
- docs/debate-platform-v2-checklist.md
- docs/demo-checklist.md
- SETUP.md
- bantr/backend/app and bantr/backend/agent\_worker.py
- bantr/frontend/src

### Appendix 3: Startup Commands

```
cd bantr/backend
uv run uvicorn app.main:app --reload
```

```
cd bantr/backend
uv run agent_worker.py dev
```

```
cd bantr/frontend
npm run dev
```